

National University of Computer and Emerging Sciences

Software Requirement Specifications

Autonomous Multimodal AI Proxy Agent for Automated Class Participation & Attendance

Project Team Members:

24K-1033

24K-0851

24K-0956

Supervision:

Supervisor: Ms. Maarukh

Submission Date: May 6, 2026

Document History

Version	Name	Date	Description
01.00	24K-1033, 24K-0851, 24K-0956	May 6, 2026	Initial SRS draft matching Interaction Diagram v1.0

Contents

1	Introduction	3
1.1	Purpose of Document	3
1.2	Intended Audience	3
1.3	Abbreviations	3
2	Overall System Description	3
2.1	Project Background	3
2.2	Project Scope	3
2.3	User Classes and Characteristics	4
3	External Interface Requirements	4
3.1	User Interfaces	4
3.2	Software Interfaces	4
3.3	Hardware Interfaces	4
4	System Modeling and Use Cases	5
4.1	Use Case Diagram Overview	5
4.2	Detailed Use Case Tables	5
4.2.1	UC-01: Automated Session Scheduling & Launch	5
4.2.2	UC-02: Multimodal Classroom Monitoring	6
4.2.3	UC-03: Attendance Verification & Response	6
4.2.4	UC-04: Contextual Query Processing	6
4.2.5	UC-05: API Failover Management	6
4.2.6	UC-06: Remote Telemetry & Notification	6
4.2.7	UC-07: Session Exit & Resource Cleanup	7
5	Functional Requirements	8
5.1	Session Automation and Control	8
5.2	Multimodal Perception and Response	8
5.3	Attendance and Notification	8
6	Non-Functional Requirements	8
6.1	Performance and Latency	8
6.2	Reliability and Resilience	9
6.3	Security and Data Integrity	9
6.4	Resource Management	9

1. Introduction

1.1. Purpose of Document

This document specifies the requirements for the Autonomous AI Proxy Agent. The system leverages a multimodal architecture to monitor virtual classrooms via browser automation, interpret auditory and textual cues using LLMs, and autonomously respond to instructors. By providing a session link and time, the agent performs a full "proxy" of the student's presence, ensuring attendance is marked and queries are addressed without manual intervention.

1.2. Intended Audience

- **Project Supervisor (Ms. Maarukh):** To verify the technical logic of the multimodal pipeline and exception handling modules.
- **Development Team:** To implement the orchestration between the `BotController`, `BrowserAutomation`, and AI APIs (Gemini/Groq).
- **System Architects:** To ensure the stability of the `NotificationManager` and `ExceptionHandler` during long-duration sessions.

1.3. Abbreviations

- **LLM:** Large Language Model (Gemini/Groq)
- **STT:** Speech-to-Text (WhisperAI)
- **BA:** Browser Automation (Chromium-based)
- **VAD:** Voice Activity Detection
- **JSON:** JavaScript Object Notation (for `schedule.json` configuration)

2. Overall System Description

2.1. Project Background

As online education becomes standardized, students often face scheduling conflicts. This project addresses the need for a "Digital Proxy"—an agent capable of not just joining a meeting, but actively listening for keywords (like the student's name), analyzing chat sentiment, and generating contextual responses that mimic human interaction.

2.2. Project Scope

The system architecture, as defined in the interaction diagram, encompasses:

- **Automated Orchestration:** A `Scheduler` that polls `schedule.json` to trigger `BrowserAutomation` at specific class times.

- **Multimodal Perception:** A `VoiceListener` using `WhisperAI` for real-time transcription and a `ChatAnalyzer` for monitoring textual interactions.
- **Cognitive Reasoning:** An `AIResponder` that interfaces with Gemini and Groq APIs to generate context-aware replies when triggers are met.
- **Attendance Management:** An `AttendanceManager` that evaluates confidence levels (90-95% accuracy) before sending a "Present" confirmation.
- **Remote Telemetry:** A `NotificationManager` that sends real-time alerts to the user via `TelegramBotAPI` regarding attendance status or class entry/exit.

2.3. User Classes and Characteristics

The system is designed for students who require a reliable, hands-free presence in digital environments. Users interact primarily through configuration files (`schedule.json`) and receive feedback via Telegram.

3. External Interface Requirements

3.1. User Interfaces

- **CLI / Startup Script:** The user initiates the bot via `run.bat`, which handles setup verification and credential loading.
- **Telegram Interface:** The primary mobile interface for receiving "Attendance Marked" notifications or "Class Missed" alerts.

3.2. Software Interfaces

- **Browser Automation Layer:** Uses Chromium to launch meeting URLs, toggle microphones, and manage browser captions.
- **AI Reasoning APIs:** Dual-integration with `GeminiAPI` (primary) and `GroqAPI` (fallback) for prompt-based response generation.
- **Audio Processing:** Integration with `WhisperAI` for transcribing live classroom audio streams into text for LLM analysis.

3.3. Hardware Interfaces

- **Virtual Audio Pipeline:** Requires a virtual audio driver to route system/browser output into the `VoiceListener` module.
- **Network:** Persistent high-bandwidth connection to maintain the browser session and simultaneous API calls to LLM providers.

4. System Modeling and Use Cases

4.1. Use Case Diagram Overview

The system's interactions are categorized into 7 primary functional use cases derived from the system architecture. These involve interactions between the **Student (User)**, the **Browser Automation Layer**, the **Multimodal Perception Pipeline (WhisperAI/Chat)**, and external actors like the **LLM Providers (Gemini/Groq)** and **Telegram API**.

UML Use Case Diagram: Autonomous AI Proxy Agent

- **UC-01:** Automated Session Scheduling & Launch
- **UC-02:** Multimodal Classroom Monitoring (Audio/Chat)
- **UC-03:** Attendance Verification & Response Logic
- **UC-04:** Contextual Query Processing (LLM Reasoning)
- **UC-05:** API Failover & Redundancy Management
- **UC-06:** Remote Telemetry & Telegram Notification
- **UC-07:** Session Exit & Resource Cleanup

Figure 1: Use Case Diagram showing the flow from schedule initialization to autonomous interaction and notification.

4.2. Detailed Use Case Tables

4.2.1 UC-01: Automated Session Scheduling & Launch

Use Case ID	UC-01
Primary Actor	Student (User)
Pre-conditions	<code>schedule.json</code> is configured; <code>run.bat</code> executed.
Main Flow	1. Scheduler polls the JSON file for class time. 2. BotController triggers BrowserAutomation. 3. System launches Chromium and joins meeting via URL.

4.2.2 UC-02: Multimodal Classroom Monitoring

Use Case ID	UC-02
Primary Actor	VoiceListener / ChatAnalyzer
Secondary Actor	WhisperAI
Main Flow	1. VoiceListener captures raw audio stream. 2. WhisperAI transcribes audio to text. 3. ChatAnalyzer scans meeting chat for name/keywords.

4.2.3 UC-03: Attendance Verification & Response

Use Case ID	UC-03
Primary Actor	AttendanceManager
Main Flow	1. System detects "attendance" or "name" trigger. 2. AttendanceManager evaluates trigger accuracy (90-95% confidence). 3. System types "Present" or triggers vocal confirmation.

4.2.4 UC-04: Contextual Query Processing

Use Case ID	UC-04
Primary Actor	AIResponder
Secondary Actor	Gemini / Groq API
Main Flow	1. System identifies a question directed at the student. 2. Text/Context sent to LLM. 3. LLM generates a human-like response for the meeting.

4.2.5 UC-05: API Failover Management

Use Case ID	UC-05
Primary Actor	ExceptionHandler
Main Flow	1. Primary API (Gemini) fails or returns error. 2. ExceptionHandler catches error and logs to file. 3. System automatically switches to GroqAPI to maintain session.

4.2.6 UC-06: Remote Telemetry & Notification

Use Case ID	UC-06
Primary Actor	NotificationManager
Secondary Actor	Telegram API
Main Flow	1. System confirms attendance marked or class joined. 2. NotificationManager formats a status alert. 3. User receives real-time update on Telegram mobile app.

4.2.7 UC-07: Session Exit & Resource Cleanup

Use Case ID	UC-07
Primary Actor	BotController
Post-conditions	Browser process killed; network resources released.
Main Flow	1. Exit condition met (e.g., participants ; 5 or class time elapsed). 2. BotController sends leave command. 3. System returns to polling state for next class.

5. Functional Requirements

5.1. Session Automation and Control

- **FR-01: Automated Scheduling:** The system shall poll the `schedule.json` file to identify upcoming sessions and trigger the `BotController` at the exact timestamp.
- **FR-02: Browser Orchestration:** The system must autonomously launch a Chromium instance, navigate to meeting URLs, and manage UI elements (toggling mic/camera and enabling captions).
- **FR-03: Dynamic Exit Logic:** The system shall monitor the participant count and automatically terminate the session if the count drops below 5 or if the scheduled duration plus a 5-minute buffer is reached.

5.2. Multimodal Perception and Response

- **FR-04: Real-time Audio Transcription:** The system shall utilize WhisperAI to transcribe live audio streams into text with a focus on detecting "name" or "attendance" keywords.
- **FR-05: Collaborative Chat Analysis:** The system must monitor meeting chat boxes for specific patterns and high-frequency "Present" bursts to trigger attendance responses.
- **FR-06: Dual-LLM Reasoning:** The `AIResponder` shall generate context-aware answers to teacher queries by querying GeminiAPI (primary) or GroqAPI (fallback).

5.3. Attendance and Notification

- **FR-07: Attendance Verification:** The system must evaluate triggers with an accuracy threshold of 90-95% before executing an "Attendance Marked" action.
- **FR-08: Remote Status Updates:** The system shall send real-time event logs (e.g., "Class Joined," "Attendance Marked," "Class Missed") to the user via the TelegramBotAPI.

6. Non-Functional Requirements

6.1. Performance and Latency

- **Processing Speed:** The conversion from raw audio stream to WhisperAI transcription must occur within a sub-2.0 second window to ensure response relevance.
- **API Latency:** The failover mechanism between Gemini and Groq must complete within 1.5 seconds to prevent awkward silences in the meeting.
- **Resource Footprint:** The browser automation must run efficiently on systems with 4GB to 8GB of RAM without causing audio stuttering.

6.2. Reliability and Resilience

- **Redundancy:** If the primary LLM API fails, the `ExceptionHandler` must successfully resume the query using the secondary provider without session termination.
- **Network Recovery:** The system shall attempt to rejoin the meeting automatically if a temporary network failure is detected by the `Scheduler`.
- **Logging:** All errors, API failures, and transaction timestamps must be logged to `log.txt` for post-session auditing.

6.3. Security and Data Integrity

- **Credential Isolation:** Meeting links and API keys must be loaded from local environment configurations to prevent data leakage.
- **Zero-Trust Communication:** Data sent to the `AIResponder` must be sanitized to prevent prompt injection from meeting chat participants.

6.4. Resource Management

To optimize system health, the "End all connections" protocol must be strictly enforced. Upon meeting exit, the system must kill all Chromium sub-processes and close API sockets to prevent memory leaks and unnecessary billing from LLM providers.